concept, then you'll see that calling a method on a class is the same as calling a method on an object:

```
an_object.a_method
AClass.a_class_method
```

See? The first line calls a method on an object referenced by a variable, and the second line calls a method on an object (that also happens to be a class) referenced by a constant. It's the same syntax.

But, wait—there's more. Remember how Bill told you that you've been using *Singleton Methods (114)* all along? That's really what class methods are: they're *Singleton Methods of a class*. In fact, if you compare the definition of a Singleton Method and the definition of a class method, you'll see that they're the same:

```
def obj.a_singleton_method; end
def MyClass.another_class_method; end
```

So, the syntax for defining a Singleton Method with `def` is always the same:

```
def object.method
  # Method body here
end
```

In the definition shown previously, `object` can be an object reference, a constant class name, or `self`. The syntax might look different in the three cases, but in truth the underlying mechanism is always the same. Nice design, don't you think?

You're not quite finished with class methods yet. There's a very useful and common spell that relies on class methods exclusively, and it deserves its own discussion.

Class Macros

Look at this example, coming straight from the core of Ruby.

The attr_accessor() Example

Ruby objects don't have attributes. If you want something that looks like an attribute, you have to define two *Mimic Methods (218)*, a reader and a writer:

class_definitions/attr.rb

```
class MyClass
  def my_attribute=(value)
    @my_attribute = value
  end
```